```
$class = 'Person';
$object = new $class;
// is equivalent to
$object = new Person;
```

Specifying a class that doesn't exist causes a runtime error.

Variables containing object references are just normal variables—they can be used in the same ways as other **variables**. Of particular note is that variable variables work with objects, as shown here:

```
$account = new Account;
$object = 'account'
${$object}->init(50000, 1.10);  // same as $account->init
```

Remember

You enclose a class using curly braces ({ }) … just like you do with functions.

6.2.1 Accessing Properties and Methods

Once you have an object, you can use the -> notation to access methods and properties of the object:

object->propertyname$

([... ,object->methodname([arg$

example

```
printf("Rasmus is %d years old.\n", $rasmus->age);  // property access
$rasmus->birthday();                                 // method call
$rasmus->set_age(21);                                // method call with arguments
```

Methods are functions, so they can take arguments and return a value:

```
$clan = $rasmus->family('extended');
```

Keyword

Variables are used to store information to be referenced and used by programs.

PHP does not have the concept of private and public methods or properties. That is, there's no way to specify that only the code in the class should be able to directly access a particular property or method. Encapsulation is achieved by convention—only an object's code should directly access its properties—rather than being enforced by the language itself.

You can use variable variables with property names:

```
$prop = 'age';
echo $rasmus->$prop;
```